

Guía de Estudio — Tema 05

Mónadas en TypeScript

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI — IF020 **Año:** 4° Licenciatura en Sistemas **Semana:** 4 — Clase 1 **Duración estimada de estudio autónomo:** 5-6 horas

Cómo usar esta guía

Esta guía está pensada para estudio autónomo y seguimiento de clase. Se basa en el programa de la materia y la secuencia de contenidos de la clase.

Secuencia recomendada:

1. Leer objetivos y conceptos previos.
 2. Estudiar cada bloque teórico con sus implementaciones en TypeScript y Clojure.
 3. Verificar las leyes monádicas con el código provisto.
 4. Resolver los ejercicios trabajados.
 5. Repasar glosario y errores frecuentes antes de pasar al TP.
-

Objetivos de Aprendizaje

Al finalizar, deberías poder:

| # | Nivel Bloom | Objetivo | |---|---|---| | OA-1 | Comprender | Explicar qué problema resuelven las mónadas y por qué surgen como patrón | | OA-2 | Aplicar | Identificar las tres leyes monádicas y verificarlas en código TypeScript | | OA-3 | Aplicar | Implementar desde cero Maybe, Either e IO en TypeScript | | OA-4 | Aplicar | Encadenar operaciones con flatMap/bind para pipelines funcionales | | OA-5 | Analizar | Contrastar la implementación monádica en TS con la aproximación idiomática en Clojure | | OA-6 | Analizar | Reconocer mónadas en APIs existentes: Promise, Array.flatMap, some-> |

Conceptos Previos

Antes de empezar, verificá que puedas explicar con tus palabras:

- Funciones puras e inmutabilidad (Tema 03).
- map, filter, reduce como pipeline funcional (Tema 03-04).
- Algebraic Data Types y tagged unions en TypeScript — Result<T, E> (Tema 04).
- Funciones de orden superior y composición (Tema 04).
- Lectura básica de Clojure: def, defn, let, ->, some->, colecciones inmutables (Tema 04).
- Tipos genéricos en TypeScript: <T>, <E, T>.

Si alguno de estos puntos no está sólido, repasá primero el Tema 04.

Desarrollo Teórico

Bloque 1 — El problema del encadenamiento (F-02, F-03, F-04, F-05, F-06, F-06b, F-06c, F-07)

1.1 Motivación: ¿por qué necesitamos mónadas?

El Tema 04 introdujo `Result<T, E>` para modelar éxito o fallo. Eso funciona para **una** operación. Pero en un programa real, encadenamos muchas operaciones que pueden fallar:

```
buscar usuario → obtener dirección → obtener código postal
```

Cada paso puede devolver `null` (TypeScript) o `nil` (Clojure). El código defensivo se vuelve un `if/else` anidado inmanejable.

En TypeScript — el infierno de los `null`:

```
function getPostalCode(userId: number): string | null {
  const user = findUser(userId);
  if (user === null) return null;

  const address = getAddress(user);
  if (address === null) return null;

  const postalCode = getPostalCode(address);
  if (postalCode === null) return null;

  return postalCode;
}
```

Tres operaciones → tres guardas `if (x === null) return null`. Con cinco operaciones serían cinco guardas. El patrón es siempre el mismo: "si el paso anterior produjo `null`, devolver `null`; si no, continuar".

En Clojure — el infierno del `when-let`:

```
(defn get-postal-code [user-id]
  (when-let [user (find-user user-id)]
    (when-let [address (get-address user)]
      (when-let [postal (get-postal-code address])
```

```
postal))))
```

Mismo patrón, mismo crecimiento lineal del anidamiento. Y peor: con `when-let`, si algo devuelve `nil`, no sabemos *por qué*. ¿No existe el usuario? ¿No tiene dirección? ¿Error de red?

El patrón repetido:

Aspecto	TypeScript	Clojure	--- --- ---	Valor ausente	<code>null / undefined</code>	<code>nil</code>
Guardia	<code>if (x === null) return null</code>		<code>(when-let [x ...])</code>		Anidamiento	
Crece linealmente		Crece linealmente		Razón del fallo	Perdida	Perdida
Solución		Abstracción que encadene		Abstracción que encadene		

La solución es una abstracción que codifique este patrón *una sola vez*: **la mónada**.

1.2 ¿Qué es una mónada? — la explicación intuitiva

Antes de entrar en código, entendamos la idea con una analogía.

Analogía: el sobre certificado ■

Imaginá que mandás una carta por correo certificado:

1. **Meter en el sobre** (`of`): ponés tu carta dentro del sobre certificado.
2. **Abrir, procesar, reenviar** (`flatMap`): el cartero abre el sobre, procesa el contenido con alguna operación, y lo mete en un nuevo sobre certificado.
3. **Si el sobre viene vacío**, el cartero no hace nada — pasa el sobre vacío al siguiente. No pregunta por qué está vacío, no intenta inventar contenido. Simplemente lo propaga.

Eso es una mónada: **una caja que envuelve un valor y agrega un contexto**. El contexto puede ser:

- **"Puede no haber valor"** → Maybe (la caja puede estar vacía)
- **"Puede haber un error con información"** → Either (la caja puede contener un mensaje de error en vez del valor)
- **"Es un efecto pendiente"** → IO (la caja contiene una receta para ejecutar después)

La regla clave: el valor de adentro no se toca directamente — se usa `flatMap` para operar sobre él. Esto parece una restricción, pero es la fuente del poder: la mónada maneja el caso de fallo **automáticamente**, sin que escribas un solo `if`.

En una frase:

Una mónada es un patrón que envuelve valores en un contexto y permite encadenar operaciones sobre ese contexto, manejando automáticamente los casos de fallo.

Antes vs después — el mismo código:

```
// ■ Sin mónadas: 3 operaciones → 3 guardas manuales
const user = findUser(1);
if (user === null) return null;
const address = getAddress(user);
if (address === null) return null;
const postal = getPostalCode(address);
if (postal === null) return null;
return postal;

// ■ Con mónadas: 3 operaciones → 0 guardas
const postal = flatMap(
  flatMap(findUser(1), getAddress),
  getPostalCode
);
```

¿Qué cambió? La lógica de "si falló, propagá el fallo" está codificada *una sola vez* dentro de `flatMap`. No la repetimos en cada paso.

1.3 De map a flatMap

En el Tema 04, `map` transforma un valor dentro de un contexto:

```
// map sobre Maybe<User>
map(just(user), u => u.name) // → Just("Ana")
```

Pero, ¿qué pasa si la función de transformación *también* devuelve un `Maybe`?

```
// findAddress devuelve Maybe<Address>, no Address
map(just(user), findAddress) // → Just(Just(address)) ¡doble envoltorio!
```

`map` mete el resultado dentro del contexto existente. Si el resultado ya es un contexto, quedan dos niveles: `Maybe<Maybe<Address>>`.

`flatMap` resuelve esto: aplica la función y aplana un nivel.

```
map:      Maybe<User> → (User → Maybe<Address>) → Maybe<Maybe<Address>> ■
flatMap: Maybe<User> → (User → Maybe<Address>) → Maybe<Address>      ■
```

Definición de trabajo:

Una **mónada** es un tipo (o protocolo, en Clojure) con dos operaciones:

1. `of / return`: mete un valor en el contexto.
2. `flatMap / bind / >>=`: encadena una función que *ya devuelve* un contexto, y aplana el resultado.

La diferencia entre `map` y `flatMap` es una línea de código. Pero esa línea es lo que permite componer pipelines enteros sin `if/nullable` anidados.

2.1 Maybe en TypeScript — Implementación completa

`Maybe<T>` modela un valor que puede o no existir. Es el reemplazo tipado de `null`.

Definición del tipo:

```
type Maybe<T> =  
  | { tag: 'Just'; value: T }  
  | { tag: 'Nothing' };
```

Tagged union discriminada por `tag`. TypeScript sabe que:

- Si `tag === 'Just'`, `.value` existe y tiene tipo `T`.
- Si `tag === 'Nothing'`, `.value` no existe.

Constructores:

```
const just = <T>(value: T): Maybe<T> =>  
  ({ tag: 'Just', value });  
  
const nothing = <T>(): Maybe<T> =>  
  ({ tag: 'Nothing' });
```

Operaciones monádicas:

```
// of: mete un valor en el contexto Maybe  
const of = <T>(value: T): Maybe<T> => just(value);  
  
// map: transforma el valor interno (si existe)  
const map = <T, U>(m: Maybe<T>, f: (x: T) => U): Maybe<U> =>  
  m.tag === 'Just' ? just(f(m.value)) : nothing();  
  
// flatMap: encadena una función que devuelve Maybe  
const flatMap = <T, U>(m: Maybe<T>, f: (x: T) => Maybe<U>): Maybe<U> =>  
  m.tag === 'Just' ? f(m.value) : nothing();
```

Comparemos `map` y `flatMap` línea a línea:

| Operación | Sobre Just | Sobre Nothing | |---|---|---| | `map(m, f)` |
`just(f(m.value))` — envuelve resultado en Just | `nothing()` | | `flatMap(m, f)` |
`f(m.value)` — `f` ya devuelve Maybe, no envuelve más | `nothing()` |

La única diferencia: `map` envuelve con `just(...)`. `flatMap` no envuelve — delega en `f`.

Pipeline completo:

```
type User    = { name: string; addressId: number };
type Address = { street: string; postalCode: string };

const findUser  = (id: number): Maybe<User>  =>
  id === 1 ? just({ name: "Ana", addressId: 10 }) : nothing();

const getAddress = (u: User): Maybe<Address>  =>
  u.addressId === 10
    ? just({ street: "San Martín 450", postalCode: "9410" })
    : nothing();

const getPostal  = (a: Address): Maybe<string> => just(a.postalCode);

// Sin flatMap: (versión con guardas)
// 12 líneas, 3 if/null

// Con flatMap:
const result = flatMap(flatMap(findUser(1), getAddress), getPostal);
// → Just("9410")

const result2 = flatMap(flatMap(findUser(99), getAddress), getPostal);
// → Nothing (findUser devuelve Nothing → todo el pipeline devuelve Nothing)
```

Si `findUser(99)` devuelve `nothing()`, los dos `flatMap` subsecuentes propagan `nothing()` sin ejecutar `getAddress` ni `getPostal`. Es el **cortocircuito automático**.

2.2 Maybe en Clojure — el enfoque idiomático con `some->`

Clojure no necesita una estructura `Just/Nothing` porque ya tiene `nil`:

```
(defn find-user [id]
  (get {1 {:name "Ana" :address-id 10}} id)) ; nil si no existe

(defn get-address [user]
  (get {10 {:street "San Martín 450" :postal-code "9410"}}
    (:address-id user)))

(defn get-postal [address]
  (:postal-code address))
```

```
;; Pipeline con some->
(some-> 1
  find-user
  get-address
  get-postal)
;; => "9410"

(some-> 99
  find-user      ; nil – corta acá
  get-address    ; no se ejecuta
  get-postal)    ; no se ejecuta
;; => nil
```

`some->` pasa el resultado de cada expresión a la siguiente como primer argumento. Si cualquier paso devuelve `nil`, el macro corta y devuelve `nil`.

Es un **Maybe implícito**: `nil` es `Nothing`, el valor no-`nil` es `Just`, y `some->` es `flatMap`. Sin tipos, sin wrapper, sin librería.

Limitación: `some->` solo funciona con `nil` como señal de ausencia. No puede distinguir "usuario no encontrado" de "error de conexión". Para eso necesitamos `Either`.

2.3 Maybe en Clojure — explícito con `cats`

La librería `cats` (`funcool`) proporciona `Maybe` formal:

```
(require '[cats.monad.maybe :as m])
(require '[cats.core :as mc])

;; Constructores
(m/just 42)      ;=> #<Just 42>
(m/nothing)      ;=> #<Nothing>

;; Funciones que devuelven Maybe
(defn find-user-m [id]
  (if-let [u (find-user id)]
    (m/just u)
    (m/nothing)))

;; Pipeline con mlet (do-notation)
(mc/mlet [user      (find-user-m 1)
          address    (get-address-m user)
          postal     (get-postal-m address)]
  (mc/return postal))
;; => #<Just "9410">

;; Equivalente con bind explícito
(mc/bind (find-user-m 1)
```

```
(fn [user]
  (mc/bind (get-address-m user)
    (fn [address]
      (get-postal-m address))))))
```

`mlet` es syntactic sugar sobre `bind` encadenado. Cada línea puede fallar con `(m/nothing)` y todo el bloque cortocircuita.

¿Cuándo usar cuál?

| Criterio | `some->` | `cats/maybe` | `|---|---|---` | Producción Clojure | ■ Idiomático | Poco usado | | Enseñanza del patrón | Implícito | ■ Explícito y formal | | Verificación de leyes | No aplicable | ■ Verificable | | Ergonomía | ■ Cero ceremonia | Más boilerplate |

2.4 Comparativa Maybe: TS vs Clojure

| Aspecto | TypeScript | Clojure | `|---|---|---` | Representación | Tagged union
Just/Nothing | `nil` nativo o `cats/maybe` | | Type safety | Compilador fuerza manejar ambos casos | Runtime — no fuerza | | Encadenamiento | `flatMap(m, f)` / `pipe` | `some->` o `(mc/bind v f)` | | Ergonomía | Verboso pero seguro | Conciso pero sin red de tipos | | Idiomático | Sí (`fp-ts Option, Effect`) | `some->` sí; `cats` nicho |

Bloque 3 — Either: la mónada del error tipado (F-15 a F-22)

3.1 Either en TypeScript — Implementación completa

`Either<E, T>` modela una computación que puede producir un valor (`Right`) o un error tipado (`Left`). Es el `Result<T, E>` del Tema 04 con nombre estándar.

Definición del tipo:

```
type Either<E, T> =
  | { tag: 'Left'; error: E }
  | { tag: 'Right'; value: T };

const left = <E, T>(error: E): Either<E, T> =>
  ({ tag: 'Left', error });

const right = <E, T>(value: T): Either<E, T> =>
  ({ tag: 'Right', value });
```

Convención universal: "right is right" — `Right` es el camino correcto.

Operaciones monádicas:

```
const of = <E, T>(value: T): Either<E, T> => right(value);

const map = <E, T, U>(
```

```

    m: Either<E, T>, f: (x: T) => U
  ): Either<E, U> =>
    m.tag === 'Right' ? right(f(m.value)) : m;

const flatMap = <E, T, U>(
  m: Either<E, T>, f: (x: T) => Either<E, U>
): Either<E, U> =>
  m.tag === 'Right' ? f(m.value) : m;

```

Sobre Left: flatMap devuelve el mismo Left sin ejecutar f. El error se **propaga intacto** a través de toda la cadena.

Ejemplo: validador de formulario

```

type ValidationError = { field: string; message: string };

const validateName = (name: string): Either<ValidationError, string> =>
  name.length >= 2
    ? right(name)
    : left({ field: 'name', message: 'Mínimo 2 caracteres' });

const validateEmail = (email: string): Either<ValidationError, string> =>
  email.includes('@')
    ? right(email)
    : left({ field: 'email', message: 'Email inválido' });

const validateAge = (age: number): Either<ValidationError, number> =>
  age >= 18
    ? right(age)
    : left({ field: 'age', message: 'Debe ser mayor de edad' });

// Pipeline: falla en la PRIMERA validación inválida
const validateForm = (name: string, email: string, age: number) =>
  flatMap(validateName(name), validName =>
    flatMap(validateEmail(email), validEmail =>
      map(validateAge(age), validAge =>
        ({ name: validName, email: validEmail, age: validAge })
      )
    )
  );

// Con datos válidos:
validateForm("Ana", "ana@mail.com", 21);
// → Right({ name: "Ana", email: "ana@mail.com", age: 21 })

// Con email inválido:
validateForm("Ana", "ana", 21);
// → Left({ field: "email", message: "Email inválido" })
// validateAge NO se ejecutó – el error se propagó

```

Either vs try/catch:

| Aspecto | try/catch | Either<E, T> | |---|---|---| | Flujo de error | Implícito (salta stack frames) | Explícito (dato en el tipo) | | Tipo del error | unknown / any | E es genérico verificable | | Composición | No compone — rompe el pipeline | flatMap compone naturalmente | | Exhaustividad | Compilador no verifica | switch sobre tag es exhaustivo | | Cuándo preferir | Errores excepcionales (disco lleno, OOM) | Errores del dominio (validación, not found) |

3.2 Either en Clojure — con cats

```
(require '[cats.monad.either :as e])
(require '[cats.core :as mc])

;; Constructores
(e/right "dato válido")  ;=> #<Right "dato válido">
(e/left "error: vacío")   ;=> #<Left "error: vacío">

;; Validaciones
(defn validate-name [name]
  (if (>= (count name) 2)
    (e/right name)
    (e/left {:field :name :msg "Mínimo 2 caracteres"})))

(defn validate-email [email]
  (if (clojure.string/includes? email "@")
    (e/right email)
    (e/left {:field :email :msg "Email inválido"})))

;; Pipeline con mlet
(mc/mlet [name (validate-name "Ana")
          email (validate-email "ana@mail.com")]
  (mc/return {:name name :email email}))
;; => #<Right {:name "Ana", :email "ana@mail.com"}>

;; Con error:
(mc/mlet [name (validate-name "A")           ; falla → Left
          email (validate-email "ana@mail")] ; no se ejecuta
  (mc/return {:name name :email email}))
;; => #<Left {:field :name, :msg "Mínimo 2 caracteres"}>
```

mlet cortocircuita igual que flatMap en TypeScript: la primera línea que devuelve Left interrumpe todo el bloque.

3.3 Either en Clojure — idiomático sin cats

La forma más común en producción Clojure — mapas con convention keywords:

```

(defn validate-name [name]
  (if (>= (count name) 2)
    {:ok name}
    {:error {:field :name :msg "Mínimo 2 chars"}}))

(defn validate-email [email]
  (if (clojure.string/includes? email "@")
    {:ok email}
    {:error {:field :email :msg "Email inválido"}}))

;; Encadenamiento manual
(defn validate-form [name email]
  (let [r1 (validate-name name)]
    (if (:error r1)
      r1
      (let [r2 (validate-email email)]
        (if (:error r2)
          r2
          {:ok {:name (:ok r1) :email (:ok r2)}}))))))

```

Funciona, pero:

- Sin `flatMap` genérico, el encadenamiento es manual y el anidamiento crece.
- Sin tipos, el compilador no verifica si manejaste todos los casos.
- Sin convención estricta, un equipo usa `:ok/:error`, otro usa `:result/:failure`. Bugs silenciosos.

Es un Either reimplementado ad-hoc, con menos ergonomía y sin garantías formales.

3.4 Comparativa Either: TS vs Clojure

| Aspecto | TypeScript | Clojure | |---|---|---| | Representación | Tagged union
Left/Right | cats/either o mapa {ok/:error} | | Error tipado | E es genérico —
compilador verifica | String, keyword o mapa — runtime | | Encadenamiento | flatMap
con tipo inferido | mlet (cats) o let+if manual | | vs try/catch | Reemplaza
completamente | Complementa (ex-info sigue siendo común) | | Exhaustividad |
Compilador verifica | Programador verifica | | Idiomático | Sí (fp-ts Either, Effect) |
cats nicho; mapas convencionales más común |

Bloque 4 — IO: la mónada de los efectos (F-23 a F-26)

4.1 IO en TypeScript

`IO<T>` encapsula un efecto (leer entrada, escribir en pantalla, acceder a red) como un valor puro. Nada se ejecuta hasta llamar `.run()`.

Definición:

```
type IO<T> = { run: () => T };

const ioOf = <T>(value: T): IO<T> =>
  ({ run: () => value });

const ioMap = <T, U>(io: IO<T>, f: (x: T) => U): IO<U> =>
  ({ run: () => f(io.run()) });

const ioFlatMap = <T, U>(io: IO<T>, f: (x: T) => IO<U>): IO<U> =>
  ({ run: () => f(io.run()).run() });
```

`IO<T>` no contiene el valor — contiene la *receta* para obtenerlo. Es un thunk tipado.

Pipeline IO:

```
const readLine: IO<string> =
  ({ run: () => prompt("Ingrese su nombre:") ?? "" });

const greet = (name: string): IO<string> =>
  ({ run: () => {
    const msg = `Hola, ${name.toUpperCase()}!`;
    console.log(msg);
    return msg;
  }});

// Composición: nada se ejecuta
const program: IO<string> = ioFlatMap(readLine, greet);

// Ejecución: un solo punto de efectos
program.run(); // Ahora sí: prompt + console.log
```

La línea `const program = ioFlatMap(readLine, greet)` no tiene efectos. Es composición pura. Solo `program.run()` dispara la cadena.

¿Por qué importa?

- Separar "qué hacer" de "cuándo hacerlo" mejora la capacidad de prueba: podemos inspeccionar y componer programas sin ejecutarlos.
- Es el principio de diseño de Haskell: todo I/O está en la mónada IO, y el `main` es la única función que ejecuta.

4.2 IO en Clojure — ¿necesaria?

Respuesta corta: **no**. Clojure es impuro por defecto, y esa es una decisión de diseño deliberada.

```
;; Efecto directo – idiomático
(println "Hola") ; se ejecuta inmediatamente, sin wrapper

;; Diferir con closures (thunks)
(def read-input (fn [] (read-line))) ; no se ejecuta hasta invocar
(read-input) ; ahora sí

;; Diferir con delay (evaluación lazy de un solo uso)
(def greeting (delay (str "Hola, " (read-line))))
@greeting ; force: ejecuta y cachea resultado

;; core.async: composición de efectos asíncronicos
(require '[clojure.core.async :refer [go <! >! chan]])
(def c (chan))
(go (>! c (str "Hola, " (<! (go (read-line))))))
```

- `delay` permite diferir, pero no compone como IO. Es un thunk de un solo uso.

- `core.async/go` blocks son lo más cercano a composición de efectos: canales como piping de valores asíncronos.
- Contraste: TypeScript necesita IO para separar puro de impuro. Clojure controla efectos por **convención**: funciones con `!` en el nombre tienen efectos (ej: `swap!`, `send!`).

Comparativa IO:

Aspecto	TypeScript	Clojure	--- --- ---	Necesidad	Alta — separa puro de impuro
Baja	lenguaje pragmáticamente impuro			Representación	<code>{ run: () => T }</code>
Thunks, <code>delay</code> , closures				Ejecución diferida	<code>.run()</code> explícito <code>@delay</code> o invocación del thunk
				Composición de efectos	<code>ioFlatMap</code> <code>core.async / go blocks</code>
				Filosofía	Pureza por tipo Control por convención

Bloque 4 (cont.) — Las tres leyes monádicas (F-27 a F-29)

Toda mónada debe cumplir tres leyes. No son opcionales: si no se cumplen, `flatMap` puede producir resultados inesperados al refactorizar.

Ley 1 — Identidad izquierda

```
of(a).flatMap(f) === f(a)
```

Envolver un valor con `of` y aplicar `flatMap(f)` es lo mismo que aplicar `f` directamente. `of` no agrega comportamiento.

Ley 2 — Identidad derecha

```
m.flatMap(of) === m
```

Aplicar `flatMap` con la función que solo envuelve (`of`) recupera el valor original. `of` es neutra.

Ley 3 — Asociatividad

```
m.flatMap(f).flatMap(g) === m.flatMap(x => f(x).flatMap(g))
```

El orden de agrupación de los `flatMap` no importa. Esto permite **refactorizar pipelines con confianza**: mover paréntesis no cambia el resultado.

Verificación en TypeScript:

```
// Ley 1: identidad izquierda
const f = (x: number): Maybe<number> => just(x * 2);
const a = 5;
console.assert(
  JSON.stringify(flatMap(of(a), f)) === JSON.stringify(f(a)),
  "Ley 1 – identidad izquierda"
);

// Ley 2: identidad derecha
const m = just(10);
console.assert(
  JSON.stringify(flatMap(m, of)) === JSON.stringify(m),
  "Ley 2 – identidad derecha"
);

// Ley 3: asociatividad
const g = (x: number): Maybe<string> => just(`val: ${x}`);
const lhs = flatMap(flatMap(m, f), g);
const rhs = flatMap(m, x => flatMap(f(x), g));
console.assert(
  JSON.stringify(lhs) === JSON.stringify(rhs),
  "Ley 3 – asociatividad"
);
// Las tres pasan ✓
```

Verificación en Clojure (REPL):

```
(require '[cats.monad.maybe :as m])
(require '[cats.core :as mc])

;; Ley 1
(let [f (fn [x] (m/just (* x 2)))
      a 5]
  (assert (= (mc/bind (mc/return m/context a) f) (f a))))

;; Ley 2
(let [mv (m/just 10)]
  (assert (= (mc/bind mv mc/return) mv)))

;; Ley 3
(let [mv (m/just 10)
      f (fn [x] (m/just (* x 2)))
      g (fn [x] (m/just (str "val: " x)))]
  (assert (= (mc/bind (mc/bind mv f) g)
             (mc/bind mv (fn [x] (mc/bind (f x) g))))))
```

Nota: `some->` no es una mónada formal — es un macro. No podemos verificar asociatividad de un macro. `cats/maybe` sí cumple las leyes porque está modelada como mónada.

Bloque 4 (cont.) — Mónadas "escondidas" (F-30 a F-31)

Promise como (casi) mónada

```
// Promise.resolve = of
const p = Promise.resolve(42);

// .then = flatMap (cuando devolvemos otra Promise)
p.then(x => Promise.resolve(x * 2))
  .then(x => console.log(x)); // 84
```

- `Promise.resolve(v) = of(v)` — envuelve un valor en el contexto de asincronía.
- `.then(f) = flatMap(f)` cuando `f` devuelve Promise. JavaScript aplana automáticamente.
- ¿Por qué "casi"? Promise es **eager**: se ejecuta al crearla. IO es **lazy**: se ejecuta con `.run()`. Promise viola la pureza, pero en la práctica cumple las leyes.

Patrones monádicos en APIs comunes

API	`of` equivalente	`flatMap` equivalente	Efecto	`Promise<T>`	
`Promise.resolve(v)`	`.then(f)`	Asincronía	`Array<T>`	`[v]`	`.flatMap(f)`
No-determinismo (múltiples resultados)	`?.` optional chaining	valor presente			
`x?.prop`	Opcionalidad (Maybe implícito)	`some->` (Clojure)	valor no-nil	threading	
que corta en nil	Maybe implícito	`for` (Clojure)	`[v]`	comprehension	List monad
`go blocks` (Clojure)	canal con valor	`<!` dentro de `go`	Async/IO		

Bloque 5 — Ecosistemas industriales (F-33 a F-34)

fp-ts y Effect (TypeScript)

No necesitamos implementar Maybe y Either a mano en producción. Estas librerías lo industrializan:

fp-ts (2019+):

```
import { pipe } from 'fp-ts/function';
import * as O from 'fp-ts/Option';

const result = pipe(
  O.fromNullable(findUser(1)),
  O.flatMap(u => O.fromNullable(getAddress(u))),
  O.map(a => a.postalCode),
  O.getOrElse(() => "N/A")
);
```

Effect (2023+):

```
import { Effect, pipe } from 'effect';

const program = pipe(
  Effect.tryPromise(() => fetch('/api/user/1')),
  Effect.flatMap(res => Effect.tryPromise(() => res.json())),
  Effect.map(user => user.name)
);
```

- fp-ts: basada en Haskell. Madura, bien documentada.
- Effect: do-notation moderna, concurrencia built-in, tipado de errores y dependencias. Dirección de la industria.

cats y manifold (Clojure)

```
;; cats: Maybe, Either, State, Writer, Reader
(require '[cats.core :as mc])

;; manifold: promesas composicionales
(require '[manifold.deferred :as d])
(def resultado (d/chain (d/future (fetch-data))
                        parse-json
                        extract-name))
```

Adopción menor que fp-ts/Effect. La comunidad Clojure prefiere datos simples.

Bloque 5 (cont.) — Mónadas para IA (F-35)

El pipeline de un agente IA es un caso real de composición monádica:

```
prompt → LLM → parseo → validación → respuesta
```

Cada paso puede fallar: prompt malformado, timeout, JSON inválido, respuesta rechazada.

En TypeScript con Either:

```
const pipeline =
  flatMap(buildPrompt(input), prompt =>
    flatMap(callLLM(prompt), raw =>
      flatMap(parseJSON(raw), parsed =>
        validateResponse(parsed)
      )
    )
  );
```

En Clojure con go + mapas:

```
(go
  (let [prompt (<! (build-prompt input))
        raw    (when (:ok prompt) (<! (call-llm (:ok prompt))))
        parsed (when raw (parse-json raw))]
    (if parsed
      (validate-response parsed)
      {:error "Pipeline falló"})))
```

El flatMap del validador de formulario es **el mismo** que compone un pipeline de IA. Es el patrón más general para composición con efectos.

Jerarquía: Functor → Monad (F-32)

Las tres abstracciones forman una jerarquía de capacidades:

| Nivel | Operación | Capacidad | |---|---|---| | **Functor** | `map` | Transformar valor dentro del contexto | | **Applicative** | `ap` | Aplicar función envuelta a valor envuelto | | **Monad** | `flatMap/bind` | Encadenar operaciones que producen contexto |

Cada nivel agrega poder: toda Monad es Applicative, todo Applicative es Functor.

En Haskell (contraste notacional — no necesitamos aprenderlo, pero conecta con la literatura):

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  return :: a -> m a
  (>=>) :: m a -> (a -> m b) -> m b -- esto es flatMap
```

Ejemplos Trabajados

Ejemplo 1 — Pipeline Maybe con datos reales

Problema: obtener el teléfono del tutor de un alumno.

```
type Student = { name: string; tutorId: number };
type Tutor    = { name: string; phone: Maybe<string> };

const findStudent = (id: number): Maybe<Student> =>
  id === 1 ? just({ name: "Luis", tutorId: 5 }) : nothing();

const findTutor = (id: number): Maybe<Tutor> =>
  id === 5 ? just({ name: "María", phone: just("2901-555-1234") }) : nothing();

const getTutorPhone = (studentId: number): Maybe<string> =>
  flatMap(findStudent(studentId), student =>
    flatMap(findTutor(student.tutorId), tutor =>
      tutor.phone // ya es Maybe<string>
    )
  );

getTutorPhone(1); // → Just("2901-555-1234")
getTutorPhone(99); // → Nothing
```

En Clojure con `some->`:

```
(some-> 1
  find-student
  :tutor-id
  find-tutor
  :phone)
;; => "2901-555-1234" o nil
```

Ejemplo 2 — Validación encadenada con Either

Problema: validar datos de inscripción a materia.

```
type InscriptionError = { field: string; msg: string };

const validateMatricula = (m: string): Either<InscriptionError, string> =>
  /^\d{5}$/.test(m)
  ? right(m)
  : left({ field: 'matricula', msg: 'Debe tener 5 dígitos' });

const validateCuatrimestre = (c: number): Either<InscriptionError, number> =>
  c >= 1 && c <= 10
  ? right(c)
  : left({ field: 'cuatrimestre', msg: 'Fuera de rango' });

const validateCorrelativas = (aprobadas: string[], requeridas: string[]): Either<InscriptionError, string> =>
  const faltantes = requeridas.filter(r => !aprobadas.includes(r));
  return faltantes.length === 0
  ? right(aprobadas)
  : left({ field: 'correlativas', msg: `Faltan: ${faltantes.join(', ')} ` });

const inscribir = (mat: string, cuat: number, aprob: string[], req: string[]) =>
  flatMap(validateMatricula(mat), m =>
    flatMap(validateCuatrimestre(cuat), c =>
      map(validateCorrelativas(aprob, req), _ =>
        ({ matricula: m, cuatrimestre: c, estado: 'inscripto' })
      )
    )
  );
```

Ejemplo 3 — IO composicional

Problema: programa que lee nombre y edad, los valida, y saluda si es válido.

```
const readAge: IO<number> =
  ({ run: () => parseInt(prompt("Edad:") ?? "0") });

const validateAndGreet = (name: string, age: number): IO<string> =>
  age >= 18
  ? ({ run: () => { const m = `Bienvenido, ${name} (${age})`; console.log(m); return m; } })
```

```

      : ({ run: () => { const m = `Menor de edad: ${name}`; console.log(m); return m; }

const program: IO<string> = ioFlatMap(readLine, name =>
  ioFlatMap(readAge, age =>
    validateAndGreet(name, age)
  )
);

// Nada se ejecutó hasta acá
program.run(); // Ahora sí: prompt + prompt + console.log

```

Glosario

| Término | Definición | |---|---| | **Mónada** | Tipo (o protocolo) con `of` y `flatMap` que cumple tres leyes | | **of / return** | Operación que envuelve un valor en el contexto monádico | | **flatMap / bind / >=>** | Operación que encadena una función que produce contexto, aplanando un nivel | | **map** | Operación de Functor que transforma el valor dentro del contexto sin aplanarlo | | **Maybe<T>** | Mónada de opcionalidad: `Just(valor)` o `Nothing` | | **Either<E, T>** | Mónada de error tipado: `Right(valor)` o `Left(error)` | | **IO<T>** | Mónada de efectos: envuelve una computación que se ejecuta con `.run()` | | **Tagged union** | Tipo TypeScript con discriminante literal (tag) que permite pattern matching seguro | | **Cortocircuito** | Comportamiento de `flatMap` que propaga `Nothing/Left` sin ejecutar la función | | **Identidad izquierda** | Ley: `of(a).flatMap(f) === f(a)` | | **Identidad derecha** | Ley: `m.flatMap(of) === m` | | **Asociatividad** | Ley: `m.flatMap(f).flatMap(g) === m.flatMap(x => f(x).flatMap(g))` | | **Functor** | Tipo con `map` — transforma valor en contexto | | **Applicative** | Tipo con `ap` — aplica función envuelta a valor envuelto | | **fp-ts** | Librería TypeScript para FP: `Option`, `Either`, `TaskEither`, `pipe` | | **Effect** | Librería moderna TypeScript: tipado de errores + dependencias + concurrencia | | **cats** | Librería Clojure (funcool) para abstracciones categóricas: `Maybe`, `Either` | | **some->** | Macro Clojure: `threading` que corta en `nil` — `Maybe` implícito | | **mlet** | Do-notation de `cats` — syntactic sugar sobre `bind` encadenado | | **core.async** | Librería Clojure: concurrencia por canales (CSP), `go blocks` | | **Thunk** | Función sin argumentos que difiere una computación: `() => efecto` |

Errores Frecuentes

| Error | Corrección | |---|---| | Confundir `map` con `flatMap` | `map` envuelve: `just(f(x)).flatMap` delega: `f(x)`. Si `f` devuelve `Maybe`, `map` anida, `flatMap` aplanar. | | Pensar que "mónada = Maybe" | `Maybe` es UNA mónada. `Either`, `IO`, `Promise`, `Array` también. Mónada es el patrón (`of` + `flatMap` + leyes). | | Creer que `Promise` es una mónada pura | Es `eager` (se ejecuta al crearla). Una mónada `IO` es `lazy`. `Promise` es "casi-mónada" en la práctica. | | Usar mónadas para todo | No toda función necesita estar en un contexto monádico. Sumar dos números no requiere `Maybe`. Usar mónadas solo cuando hay efecto o error en el dominio. | | Ignorar las

leyes | Las leyes no son formalidad académica — garantizan que refactorizar el pipeline no cambia su comportamiento. Sin la ley 3, mover paréntesis puede romper el código. | | Pensar que Clojure necesita `cats` para ser funcional | `some->`, `when-let` y mapas `{:ok/:error}` son patrones monádicos idiomáticos sin librería. `cats` es para formalidad, no para producción. |

Lecturas Complementarias

- **Wadler, P. (1995).** *Monads for Functional Programming*. Paper fundacional que explica mónadas con ejemplos en un lenguaje funcional accesible. [Disponible en [_edu-knowledge/references/monads-pdfs/](#)]
- **Anderlind & Åsberg (2023).** *Monadic Programming in Imperative Languages*. Tesis de Chalmers sobre implementación de mónadas en lenguajes imperativos (Rust y C#). Evalúa si el patrón mejora la calidad del código. [Disponible en [_edu-knowledge/references/monads-pdfs/](#)]
- **funcool/cats:** Documentación oficial — <https://funcool.github.io/cats/latest/>
- **fp-ts:** Documentación — <https://gcanti.github.io/fp-ts/>
- **Effect:** Documentación — <https://effect.website/>